



Case Study Report

on

CASE STUDY 161: GROCERY INVENTORY CALCULATOR

- **Subject: Java Programming**
- **Student Name: Ankitraj Jha**
- **Roll No: 150096725050**
- **Class: Demis Hassabis**

1. INTRODUCTION & PROBLEM STATEMENT

1.1 Introduction

In retail operations, managing grocery stock is a daily mission-critical task. Small to medium-sized grocery enterprises often struggle with manual tracking of rapidly moving goods, resulting in stock discrepancies, expired shelf life, sudden stock-outs, and unaccounted capital tied up in slow-moving inventory.

Automating inventory valuation and threshold monitoring ensures timely replenishment, financial clarity, and operational efficiency.

1.2 Problem Statement

A grocery store requires a lightweight, dependable computer application to:

1. Systematically capture product entries (name, unit price, stock quantity, and minimum required threshold).
2. Compute accurate financial inventory values for individual items and cumulative stock.
3. Automatically flag items falling below safety stock levels to prompt restocking.
4. Allow instant keyword searching for stock availability.
5. Produce a well-formatted summary inventory report.

1.3 Scope & Curriculum Alignment

Designed strictly around Semester III foundational Java core competencies, this project demonstrates mastery over:

- **Primitive Data Structures:** Multi-attribute record management using synchronized parallel 1D arrays without external collection frameworks.
- **Control Structures & Iteration:** Deterministic `for` loops and sentinel-controlled `do-while` loops driving interactive execution.
- **Conditional Branching & Logic:** Multi-way `if-else` decision trees with compound relational and logical operators (`&&`, `||`, `<`, `<=`). matching.
- **Defensive Programming:** Robust scanner input handling eliminating buffer overflows and `InputMismatchException` crashes.

2. SYSTEM REQUIREMENTS & SPECIFICATIONS

2.1 Hardware Requirements

- **Processor:** Dual-core x86/x64 or ARM-based processor (Intel Core i3+, AMD Ryzen 3+, or Apple Silicon M-series)
- **RAM:** 4 GB minimum (8 GB recommended)
- **Storage Space:** 150 MB available hard drive space
- **I/O Peripherals:** Standard keyboard, display monitor (minimum 1366×768 resolution), and console terminal

2.2 Software Environment

- **Operating System:** macOS 12+, Windows 10/11 (64-bit), or Ubuntu Linux 20.04+
- **Java Platform:** Java Development Kit (JDK 17 LTS / JDK 21 LTS / OpenJDK)
- **Compiler & VM:** javac compiler and Java Virtual Machine (`java`)
- **Development Environment:** Visual Studio Code with Java Extension Pack

3. CORE MODULE ARCHITECTURE & CODE SNIPPETS

3.1 Module 1: Data Storage Architecture (Parallel Arrays)

The application avoids dynamic heap-overhead collections (`ArrayList`) by utilizing four synchronized, fixed-size 1D arrays governed by a common capacity boundary `MAX_PRODUCTS = 100` and tracking pointer `productCount`.// Snippet: Synchronized Parallel Arrays Definition

Java

```
static final int MAX_PRODUCTS = 100;
static int productCount = 0;
static String[] names = new String[MAX_PRODUCTS];
static double[] prices = new double[MAX_PRODUCTS];
static int[] quantities = new int[MAX_PRODUCTS];
static int[] minStock = new int[MAX_PRODUCTS];
```

3.2 Module 2: Product Entry & Validation

Accepts item details from the store manager. It performs real-time duplicate validation against existing stock records, strictly rejects zero or negative prices, and issues an immediate restocking advisory if initial stock is below minimum safe threshold.// Snippet: Record insertion with validation & duplicate check

Java

```
if (nameExists(name)) {
    System.out.println(" Error: Product '" + name + "'
already exists in inventory.");
}
names[productCount] = name;
prices[productCount] = price;
quantities[productCount] = qty;
minStock[productCount] = min;
productCount++;
if (qty < min) {
    System.out.println(" [!] Note: Quantity is below the
minimum required stock (" + min + "). Restock needed.");
}
```

3.3 Module 3: Inventory Valuation Engine

Computes itemized financial asset value by multiplying unit price by units on hand (price * quantity), tabulating individual items, and accumulating the store's grand inventory valuation in Indian Rupees (Rs.).

Java

```
// Snippet: Valuation method and grand total accumulator
public static double calculateValue(double price, int
quantity) {
    return price * quantity;
}
```

Java

```
double grandTotal = 0.0;
for (int i = 0; i < productCount; i++) {
    double itemTotal = calculateValue(prices[i],
quantities[i]);
    grandTotal += itemTotal;
}
```

3.4 Module 4: Stock Validation & Restock Deficit Analyzer

Scans active inventory to isolate under-stocked items. It calculates the exact replenishment deficit (`minStock - currentQuantity`) and categorizes each item into distinct operational states: `OUT OF STOCK`, `LOW`, or `ADEQUATE`.

Java

```
// Snippet: Restock logic & multi-way status classification
public static boolean isLowStock(int quantity, int minimum)
{
    return quantity < minimum;
}
```

```

public static String getStatus(int quantity, int minimum) {
    if (quantity == 0) {
        return "OUT OF STOCK";
    } else if (quantity < minimum) {
        return "LOW";
    } else {
        return "ADEQUATE";
    }
}

```

3.5 Module 5: Search & Retrieval Engine

Executes a linear search traversal across the `names[ ]` array. It performs case-insensitive comparisons using `equalsIgnoreCase()`, terminating immediately upon encountering the match ($O(1)$ best-case, $O(n)$ worst-case).// Snippet: Optimized linear search traversal

Java

```

// Snippet: Optimized linear search traversal
int foundIndex = -1;
for (int i = 0; i < productCount; i++) {
    if (names[i].equalsIgnoreCase(query)) {
        foundIndex = i;
        break; // Early exit on match
    }
}

```

3.6 Defensive I/O: Buffer-Safe Scanner Parsing

Eliminates common Java Scanner pitfalls where `nextInt()` leaves dangling newline characters in the buffer. Uses `scanner.nextLine()` with safe parsing wrapped in structured exception handling.// Snippet: Exception-safe integer and double readers

Java

```
static int readInt(String prompt) {  
    while (true) {  
        System.out.print(prompt);  
        String input = scanner.nextLine().trim();  
        try {  
            return Integer.parseInt(input);  
        } catch (NumberFormatException e) {  
            System.out.println(" Error: Invalid number  
format. Please enter an integer.");  
        }  
    }  
}
```

4. CONSOLE OUTPUTS & SCREENSHOT PLACEMENTS

Submission Note: Paste your terminal execution screenshots into the designated placeholder boxes below before finalizing the document.

4.1 Main Menu Interface

```
ankitrajha@Ankitrajs-MacBook-Air JAVA Main Project % cd "/Users/ankitrajha/Projects/InventoryCalculator"
=====
      GROCERY INVENTORY CALCULATOR – STORE SYSTEM
=====

===== MAIN MENU =====
1. Add Products
2. Calculate Inventory Value
3. Show Low-Stock Products
4. Search Product
5. Display Inventory Report
6. Exit
Enter your choice (1-6):
```

4.2 Module 1: Product Entry & Input Validation

```
=====
      GROCERY INVENTORY CALCULATOR – STORE SYSTEM
=====

===== MAIN MENU =====
1. Add Products
2. Calculate Inventory Value
3. Show Low-Stock Products
4. Search Product
5. Display Inventory Report
6. Exit
Enter your choice (1-6): 1

--- [Module 1: Product Entry] ---
Available slots in inventory: 100
How many products would you like to add? 2

Entering details for Product #1:
Enter Product Name: rice
Enter Unit Price (Rs. ): 12121
Enter Quantity in Stock: 4
Enter Minimum Stock Threshold: 1
[OK] 'rice' added successfully!

Entering details for Product #2:
Enter Product Name: wheat
Enter Unit Price (Rs. ): 2222
Enter Quantity in Stock: 32
Enter Minimum Stock Threshold: 22
[OK] 'wheat' added successfully!

[OK] All 2 product(s) registered. Total products in store: 2
```

4.3 Module 2: Inventory Calculation Output

```
===== MAIN MENU =====
1. Add Products
2. Calculate Inventory Value
3. Show Low-Stock Products
4. Search Product
5. Display Inventory Report
6. Exit
Enter your choice (1-6):
[!] Invalid input! Please enter a valid integer number.
Enter your choice (1-6): 2

--- [Module 2: Inventory Value Calculation] ---
Inventory Breakdown (per product):
    rice                : Rs. 12121.00 x 4 = Rs. 48484.00
    wheat               : Rs. 2222.00 x 32 = Rs. 71104.00
-----
Total Distinct Products : 2
Total Inventory Valuation: Rs. 119588.00
-----
```

4.4 Module 3: Stock Validation & Low-Stock Alerts

```
===== MAIN MENU =====
1. Add Products
2. Calculate Inventory Value
3. Show Low-Stock Products
4. Search Product
5. Display Inventory Report
6. Exit
Enter your choice (1-6): 3

--- [Module 3: Stock Validation (Low-Stock Alert)] ---
No.   | Product Name          | Current   | Min Stock | Deficit Units | Stock Level
-----|-----|-----|-----|-----|-----
All products have sufficient stock. None need restocking.
-----
```

4.5 Module 4: Product Search Operation

```
Enter your choice (1-6): 4

--- [Module 4: Product Search (Linear Search)] ---
Enter the product name to search: rice

[OK] Product Found!

-----
Product Index : #1
Product Name  : rice
Unit Price    : Rs. 12121.00
Stock Quantity: 4 units
Min Threshold : 1 units
Total Value   : Rs. 48484.00
Stock Status  : OK
-----

-----
```

4.6 Module 5: Comprehensive Inventory Status Report

```
Enter your choice (1-6): 5

--- [Module 5: Inventory Report] ---

=====
No. | Name           | Price (Rs) | Quantity | Min Stock | Value (Rs) | Status
=====
1   | rice           | 12121.00   | 4        | 1         | 48484.00   | OK
2   | wheat          | 2222.00    | 32       | 22        | 71104.00   | OK
=====

Total Distinct Products : 2
Total Physical Units    : 36 units
Total Inventory Value    : Rs. 119588.00
=====

-----
```

5. EXTENSION SECTION: GRAPHICAL USER INTERFACE (SWING)

5.1 Architecture & Design Highlights

- **Architectural Decoupling:** Business calculations and parallel array state reside inside `InventoryManager.java` with zero GUI dependencies, ensuring 100% logic reusability.
- **Component Framework:** Utilizes a 5-tab `JTabbedPane`, structured `JTable` data grids, native `BorderLayout` and `GridLayout` managers, and a custom `TableCellRenderer` that dynamically renders understocked rows in **soft red**.

5.2 GUI Screenshot Placements

The screenshot displays the 'Grocery Inventory Calculator' application window. The title bar reads 'Grocery Inventory Calculator'. The main window has a dark green header with the title 'Grocery Inventory Calculator'. Below the header is a tabbed interface with five tabs: 'Product Entry' (selected), 'Inventory Calculation', 'Stock Validation', 'Product Search', and 'Inventory Report'. The 'Product Entry' tab contains four input fields labeled 'Product Name:', 'Unit Price (Rs.):', 'Quantity in Stock:', and 'Minimum Stock:'. Below these fields are two buttons: 'Add Product' (green) and 'Clear' (white). At the bottom, there is a text instruction: 'Enter details and click Add Product.'

Grocery Inventory Calculator

Product Entry

Inventory Calculation

Stock Validation

Product Search

Inventory Report

Calculate

Total Inventory Value: Rs. 266662.00

Name	Price (Rs.)	Quantity	Value (Rs.)
rice	12121.00	22	266662.00
TOTAL			266662.00

Grocery Inventory Calculator

Product Entry

Inventory Calculation

Stock Validation

Product Search

Inventory Report

Show Low-Stock Products

[!] Alert: 1 product(s) low on stock.

Name	Current	Min Stock	Deficit	Level
rice	22	30	8	LOW

Grocery Inventory Calculator

Product Entry

Inventory Calculation

Stock Validation

Product Search

Inventory Report

Product Name:

Search

Field	Details
Product Name	rice
Unit Price	Rs. 12121.00
Stock Quantity	22 units
Min Threshold	30 units
Total Value	Rs. 266662.00
Stock Status	RESTOCK

Grocery Inventory Calculator						
Product Entry Inventory Calculation Stock Validation Product Search Inventory Report						
No.	Name	Price (Rs.)	Quantity	Min Stock	Value (Rs.)	Status
1	rice	12121.00	22	30	266662.00	RESTOCK

6. CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The **Grocery Inventory Calculator** case study successfully accomplishes all academic and functional requirements set forth by ITM Skills University. By synthesizing fundamental Java building blocks—synchronized parallel arrays, iterative structures, defensive exception handling, mathematical operators, and linear searching—the project provides an accurate, dependable inventory tracking system. The design demonstrates that robust, production-quality retail

utilities can be constructed using pure foundational Java principles without heavy external dependencies.

6.2 Future Scope & Industrial Enhancements

1. **Persistent Data Storage:** Integrating standard File I/O (`java.io.File`, CSV serialization) or database connectivity via JDBC (SQLite / MySQL) to retain inventory state across application restarts.
2. **Hardware Barcode Integration:** Connecting a USB handheld barcode reader to pipe SKU numbers directly into the search and entry modules.
3. **Automated Supplier Email Alerts:** Incorporating `javax.mail` APIs to automatically draft and dispatch restock purchase orders to vendors when inventory items hit zero units.
4. **Role-Based Access Control (RBAC):** Introducing login authentication to separate cashier functions (view/search) from manager functions (add/modify stock).